Project Design Phase-II Technology Stack
(Architecture & Stack)

Date	31 January 2026
Team ID	699d5f80fdf409fdbd1bec27
Project Name	ArogyaMitra – AI-Driven Workout Planning, Nutrition Guidance, and Health Coaching Platform
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example: Order processing during pandemics for offline mode

Reference: https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/

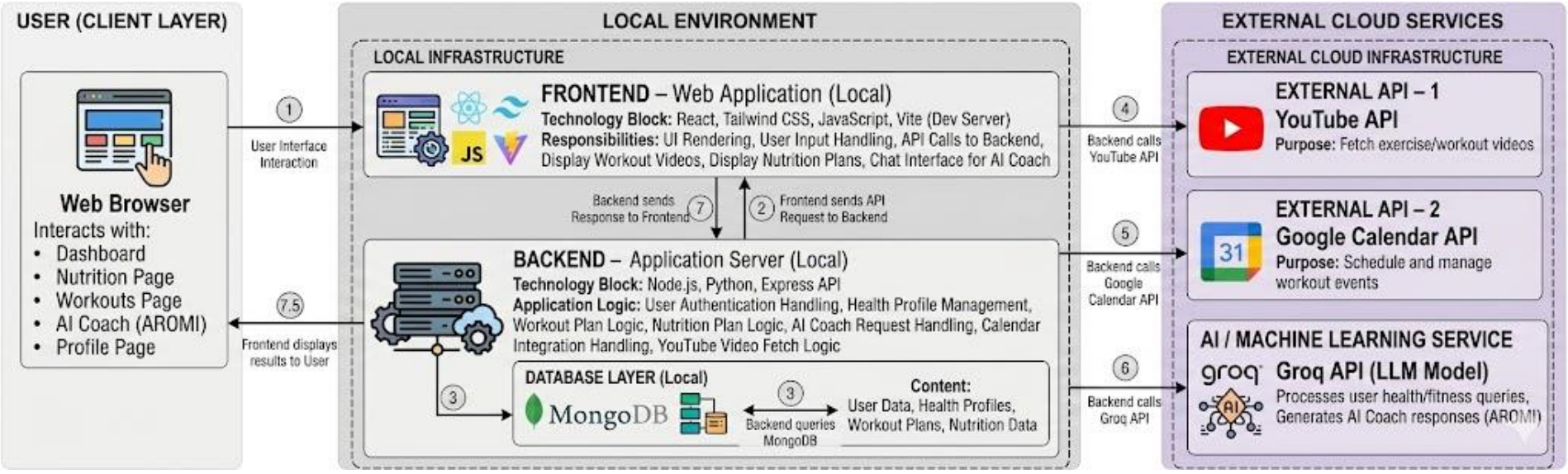

Table 1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Web-based dashboard with pages for Dashboard, Nutrition, Workouts, Profile and AI Coach interface	HTML, CSS, JavaScript, React, Tailwind CSS
2	Application Logic-1	Backend API logic for authentication, health profile management, workouts, nutrition plans, AI coach, calendar and YouTube integration	Node.js, Python
3	Application Logic-2	AI response handling and conversational logic for AROMI coach using Groq API	Groq API
4	Application Logic-3	AI wellness assistant (AROMI) for answering fitness and nutrition queries	AI service integration via Groq API
5	Database	Stores user data, health profile, workouts, nutrition plans	MongoDB
6	Cloud Database	MongoDB database service	MongoDB
7	File Storage	Local filesystem for project assets and screenshots	Local Filesystem
8	External API-1	Fetches exercise videos for workouts	YouTube API

9	External API-2	Calendar scheduling integration	Google Calendar API
10	Machine Learning Model	AI-powered conversational health assistant for personalized recommendations	Groq API (LLM-based AI model)
11	Infrastructure (Server / Cloud)	Local development server for frontend and backend	Local server (Node.js runtime, Vite dev server)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Frontend framework and styling library used in the project	React, Tailwind CSS
2	Security Implementations	Use of environment variables for API keys and backend secret key configuration	SECRET_KEY, Environment Variables
3	Scalable Architecture	Separation of frontend and backend with API-based communication	React (Frontend), Node.js / Python (Backend)

4	Availability	Application runs on local development server	Local Deployment
5	Performance	No explicit caching, CDN, or performance optimization used due to project's scale not requiring theme	Not Implemented

References:

https://c4model.com/

https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/

https://www.ibm.com/cloud/architecture

https://aws.amazon.com/architecture

https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d